NKT
Tag 4: Generics, Vererbung, Binäre Suchbäume

Alexa Katharina-Ronja Brockmöller, Tobias Wrede

5. März 2026

Gliederung

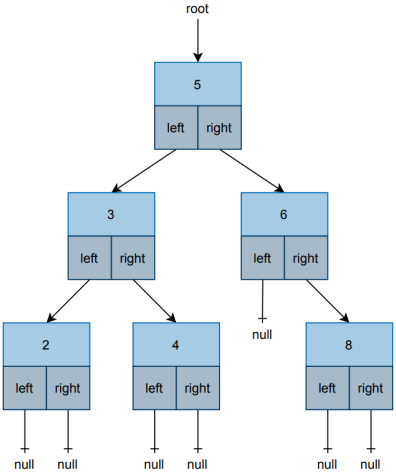
1. Binäre Bäume
   - Binärer Suchbaum
   - Einfügen
   - Minimum finden
2. Exceptions
3. Hashing
   - Was ist Hashing?
   - Hashfunktionen
   - Kollisionen
   - Hashset
   - Aufgabe: Hashfunktion

Gliederung

1. Binäre Bäume
   - Binärer Suchbaum
   - Einfügen
   - Minimum finden
2. Exceptions
3. Hashing
   - Was ist Hashing?
   - Hashfunktionen
   - Kollisionen
   - Hashset
   - Aufgabe: Hashfunktion

Bäume

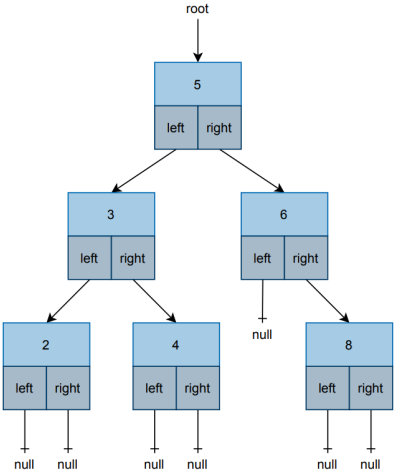
- Jedes Element (Knoten): linker und rechter Nachfolger (Kind) (= „Binärbaum“)



Binärer Suchbaum

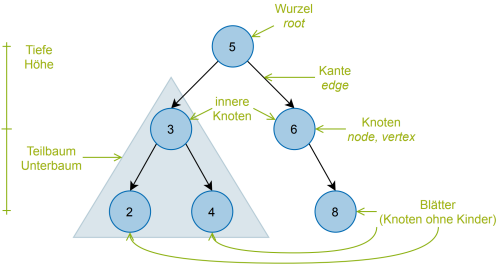
- Ordnung:
  - Alle Knoten im linken Teilbaum kleiner als (Teilbaum-) Wurzel
  - alle Knoten im rechten Teilbaum größer

  (= „binärer Suchbaum“)
- keine doppelten Werte



Bäume

- 2 ist linkes Kind von 3.
- 3 ist Elternknoten von 2 und 4.
- 3 ist die Wurzel des linken Teilbaums von 5.
- Der Baum hat die Tiefe 3.



Bäume - Umsetzung

```
class BinarySearchTree {

    private class BinaryNode {
    private int element;
    private BinaryNode left, right;


    private BinaryNode(int element) {
        this.element = element;
    }
}

private BinaryNode root;
}
```

Bäume - Einfügen (Teil 1)

Heinrich Heine Universität Düsseldorf

```
1  void insert(int newNumber) {
2      // Sonderfall: leerer Baum
3      if (root == null) {
4          root = new BinaryNode(newNumber);
5          return;
6      }
7
8      BinaryNode parent = null;
9      BinaryNode child = root;
10 [...]
```

Bäume - Einfügen (Teil 2)

Heinrich Heine Universität Düsseldorf

Baum durchsuchen

```
while (child != null) {
    parent = child;
    if (newNumber == child.element) {
        // Zahl bereits vorhanden
        return;
    } else if (newNumber < child.element) {
        child = child.left;
    } else {
        child = child.right;
    }
}
```

Bäume - Einfügen (Teil 3)

Heinrich Heine Universität Düsseldorf

Neue Node einfügen

```
if (newNumber < parent.element) {
    parent.left = new BinaryNode(newNumber);
} else {
    parent.right = new BinaryNode(newNumber);
}}
```

Bäume - Minimum finden

```
1  int minimum() {
2      if(root == null) {
3          throw new java.util.NoSuchElementException();
4      }
5
6      BinaryNode current = root;
7      while(current.left != null) {
8          current = current.left;
9      }
10
11     return current.element;
12 }
```

Gliederung

1 Binäre Bäume
- Binärer Suchbaum
- Einfügen
- Minimum finden

2 Exceptions

3 Hashing
- Was ist Hashing?
- Hashfunktionen
- Kollisionen
- Hashset
- Aufgabe: Hashfunktion

Einschub: Exceptions

```
throw new [Exception]
```

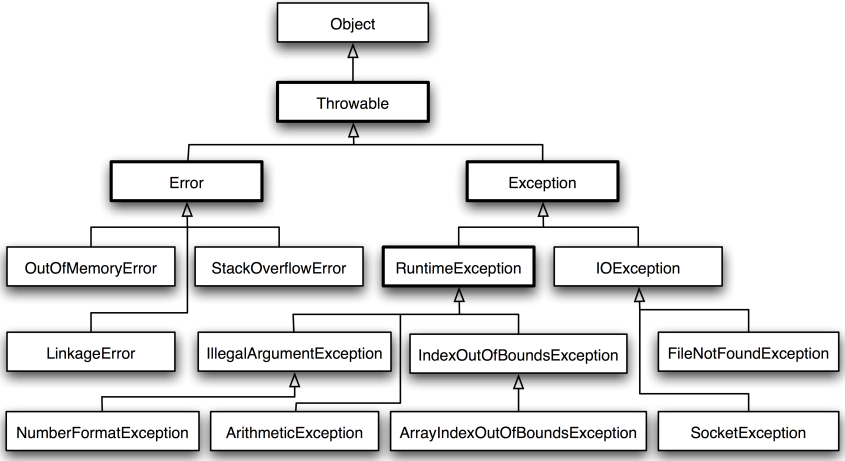
Throwing (werfen) → sorgt für das auslösen einer Exception.

```
Try{
    //Was versucht werden soll
} catch([Exceptiontyp] e) {
    // Code zur Fehlerbehandlung
}
```

- Fängt Exceptions ab
- verhindert Programmabsturz.
- Errors, können auch abgefangen werden! Ist aber fast immer eine schlechte Lösung!

Vorsicht! Alle Exceptions folgen einer Umfangreichen Vererbungshierarchie. Wenn ein zu hoher Obertyp gewählt wird, kann die tatsächliche Exception verdeckt werden oder kann nicht abgefangen werden

Einschub: Exceptions



https://www.protechtraining.com/static/bookshelf/java_fundamentals_tutorial/images/ExceptionClassHierarchy.png

Gliederung

1 Binäre Bäume
- Binärer Suchbaum
- Einfügen
- Minimum finden

2 Exceptions

3 Hashing
- Was ist Hashing?
- Hashfunktionen
- Kollisionen
- Hashset
- Aufgabe: Hashfunktion

Was ist Hashing?

Heinrich Heine Universität Düsseldorf

- Hashing ist ein Verfahren, um Daten schnell zu finden.
- Es wandelt einen Schlüssel (z.B. einen String) anhand von einer Hashfunktion in einen Index um.
- Dieser Index zeigt auf eine Position in einem Hashtabellen-Array.
- Array-Platz ‘null‘ = leer
- Ziel: contains() geht schnell

Hashfunktionen

- Eine Hashfunktion $h(k)$ nimmt einen Schlüssel k und gibt einen Index zurück.
- Gute Hashfunktionen verteilen Schlüssel gleichmäßig auf die Hashtabellen-Plätze.
- Klassen definieren selbst ‘hashCode‘ Methode, die Hashfunktion definiert.

Kollisionen

- Kollision: Zwei Schlüssel werden auf denselben Platz abgebildet.
- Lösungsstrategien:
  - Verkettung: Buckets sind Listen von Schlüsseln.
  - Sondierung: Suche nach einem anderen freien Platz.

Hashset

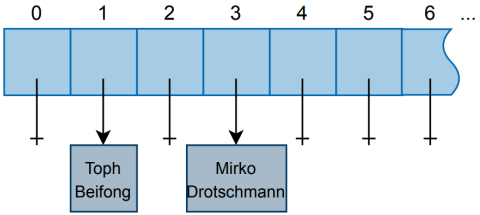
Heinrich Heine Universität Düsseldorf

- Datenstruktur, die wie Menge funktioniert
  - behält keine (sinnvolle) Sortierung
  - keine doppelten Elemente
- Grundidee: Objekte in großem Array speichern, Hashfunktion gibt Index vor

Hashset

Personen sollen häufig und schnell über Nachnamen gefunden werden

o	$h(o)$
Beifong	1
Drotschmann	3
Nguyen-Kim	13
Schönfeld	18



h: Position des ersten Buchstabens im Alphabet (minus 1)

Aufgabe 1: Hashfunktion

Heinrich Heine Universität Düsseldorf

Gegeben sei eine Hashtabelle der Größe 10 und die Hashfunktion $h(k) = k \mod 10$.

1. Berechnen Sie die Hashwerte für die Schlüssel 17, 25, 38, 42, 59.
2. Zeichnen Sie die Hashtabelle nach dem Einfügen der Schlüssel unter Verwendung von Verkettung.